

EXPERIMENT 1

QUERY PROCESSING

(a) Performing CRUD Operations (HEADING)

Q: Write a MongoDB query to create a CUSTOMERDB database and collections(CUSTOMER,) and its corresponding documents.

1. Create a collection named customers in customerDB

```
db.createCollection("customers")
```

2. Insert a single customer record with name, city, age, and phone number.

```
db.customers.insertOne({name: "Akhila", city: "Kochi", age: 25, phone: "9876543210"})
```

3. Insert multiple customers

```
db.customers.insertMany([ { name: "Kochu", city: "Trivandrum", age: 28, phone: "9123456789" }, { name: "Aswin", city: "Kollam", age: 21, phone: "9988776655" }, { name: "Kannan", city: "Kochi", age: 32, phone: "9998887776" }, { name: "Anu", city: "Kottayam", age: 24, phone: "9090909090" } ])
```

4. Write a MongoDB query to display all documents from the collection CUSTOMERS

```
db.customers.find()
```

5. Write a MongoDB query to show all documents from the collection CUSTOMER in formatted way

```
db.customers.find().pretty()
```

6. Update the age of the customer named "Kannan" to 29.

```
db.customers.updateOne({ name: "Kannan" }, { $set: { age: 29 } })
```

7. Write a MongoDB query to delete the first matched document whose name is "Aswin" and display the documents

```
db.customers.deleteOne({ name: "Aswin" })
```

```
db.customers.find().pretty()
```

EXPERIMENT 2

NoSQL – RETRIEVING DATA

1. Write a MongoDB query to display the customer documents from "Kochi" OR age less than 25.

```
db.customers.find({ $or: [ { city: "Kochi" }, { age: { $lt: 24 } } ] })
```

2. Write a MongoDB query to display all customer documents who have age < 26.

```
db.customers.find({ age: { $lt: 26 } })
```

3. Write a MongoDB query to display all customer documents who have age >= 40.

```
db.customers.find({ age: { $gte: 29 } })
```

4. Write a MongoDB query to find customer documents who not live in 'kochi'.

```
db.customers.find({ city: { $ne: "Kochi" } })
```

EXPERIMENT NO.3

MONGODB-AGGREGATE FUNCTIONS

1. Write a MongoDB query to sort customer details in ascending order of their name.

```
db.customers.find().sort({ name: 1}).pretty()
```

2. Write a MongoDB query to count number of city in CUSTOMER.

```
db.customers.aggregate([{$group: { _id: "$city" } },{$count: "total_cities"}])
```

3. Write a MongoDB query to count number of customers in each city.

```
db.customers.aggregate([{$group:{_id: "$city",total:{$sum:1}}})])
```

4. Write a MongoDB query to find minimum and maximum on CUSTOMER collection

```
db.customers.aggregate([{$group:{_id:"",min_age:{$min:"$age"},max_age:{$max:" $age"} } }])
```